

Алгоритмы и структуры данных

Универсальное пособие: билеты 1–24

Текст всех тем + схемы интерактивных визуализаций.

Собрано автоматически из сайта-пособия.

13.09.2026

Содержание

Обзор: Кратчайшие пути и Графы	3
1. Дерево отрезков с операциями на отрезках	5
2. Двумерная разреженная матрица (Sparse Table)	6
3. Декартово дерево (Treap)	7
4. Splay-дерево	10
5. Двумерная матрица префиксных сумм	14
6. Графы. Представление графов, DFS, BFS	15
7. Графы. Планарные. Покраска	17
8. Графы. Компоненты связности	20
9. Графы. Топологическая сортировка	21
10. Графы. Компоненты сильной связности	23
11. Графы. Мосты	25
12. Графы. Точки сочленения	28
13. Графы. Эйлеров цикл (и эйлеров путь)	31
14. Графы. Поиск кратчайшего пути. Дейкстра	33
15. Графы. Поиск кратчайшего пути. Форд-Беллман	34
16. Графы. Поиск кратчайшего пути. Флойд	35
17. Графы. Поиск кратчайшего пути. Ускорения (алгоритм Джонсона)	36
18. Графы. Остовное дерево. Краскал	38
19. Графы. Остовное дерево. Прима	39
20. Графы. Остовное дерево. Борувка	40
21. Алгоритм Кнута-Морриса-Пратта (префикс-функция)	41
22. Z-функция	43
23. Алгоритм Ахо-Корасик	44
24. Классы сложности, сведение задач	46

Обзор: Кратчайшие пути и Графы

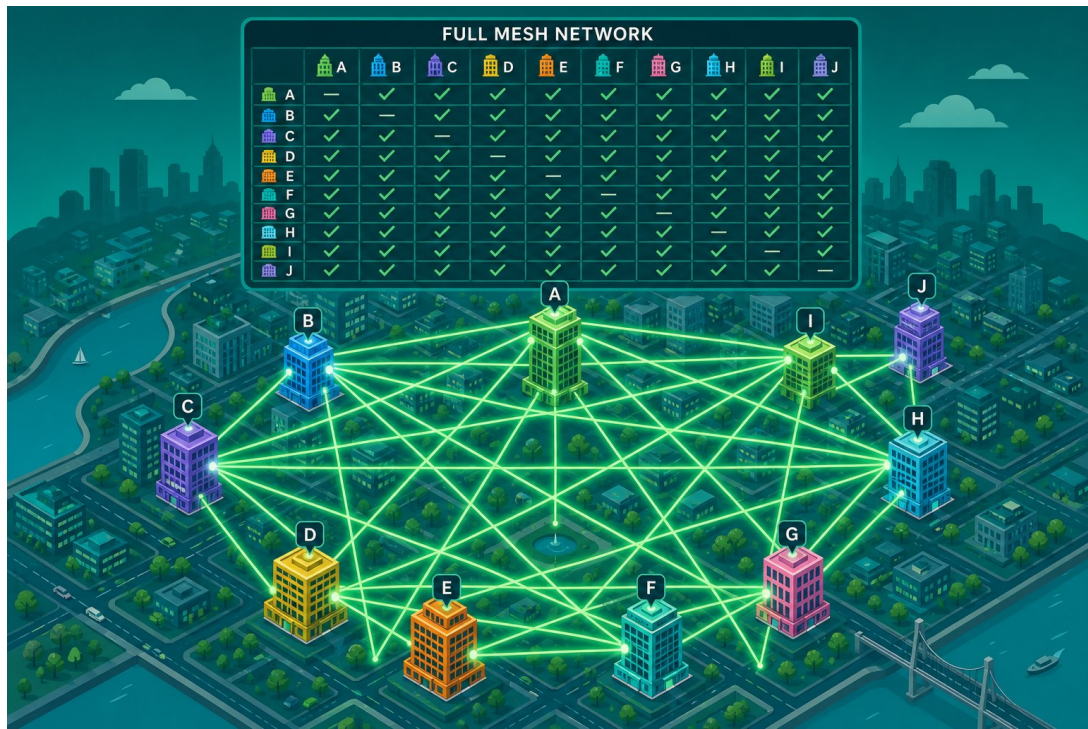
Обзор: Кратчайшие пути

Для чего это нужно?

Представь, что города — это вершины, а дороги между ними — ребра с весом (стоимостью проезда). Алгоритмы поиска кратчайшего пути показывают, как добраться из точки А в точку Б максимально дешево или быстро.

■ Визуализация: Мнемокарточки





1. Дерево отрезков с операциями на отрезках

Дерево отрезков — структура данных для запросов на отрезке за $O(\log n)$.

Дерево отрезков с операциями на отрезках

Массовое обновление (Lazy Propagation)

Строгое правило / Формула:

Не обновляй детей, пока к ним не обратятся. Храни "обещание" обновить в массиве `promise (lazy)`.

Босс решает выписать премию всем 10 000 сотрудникам компании. Вместо того, чтобы рассылать 10 000 писем, он пишет одно письмо начальникам департаментов (обещание). Начальники департаментов обновят суммы на счетах сотрудников только тогда, когда сотрудник придет в бухгалтерию спросить свой баланс. Это и есть **Lazy Propagation** (отложенное обновление).

Представь массив как одну огромную коробку. Внутри неё лежат две коробки поменьше, и так далее. Если нам нужно покрасить половину массива в синий цвет, мы не красим каждый элемент. Мы просто лепим стикер "Внутри всё синее!" на большую коробку. Опускаемся внутрь только при острой необходимости.

► **Строгое доказательство / Код (Скрыто)** — см. интерактивную версию

```
function push(node) {
  if (lazy[node] !== 0) {
    lazy[2 * node] += lazy[node];
    tree[2 * node] += lazy[node];
    lazy[2 * node + 1] += lazy[node];
    tree[2 * node + 1] += lazy[node];
    lazy[node] = 0;
  }
}
```

■ **Визуализация: Дерево отрезков**

2. Двумерная разреженная матрица (Sparse Table)

Разреженная таблица для идемпотентных запросов RMQ.

Двумерная разреженная таблица

RMQ за $O(1)$

Строгое правило / Формула:

$ST[i][j] = \min(ST[i][j-1], ST[i + 2^{(j-1)}][j-1])$. Ответ берется как пересечение двух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$.

Для идемпотентных операций (когда $x \circ x = x$, например, минимум). Тебе дают отрезок длины 7. Ты берешь две заготовленные заранее линейки длины 4 (степень двойки) и накладываешь их так, чтобы они покрыли весь отрезок длины 7 (перекрывая друг друга посередине). Минимум из этих двух линеек и будет минимумом на всём отрезке! Не нужно ничего складывать, просто пересекаем куски. Предварительно мы считаем ответы для всех отрезков длин 1, 2, 4, 8... И когда нужно покрыть отрезок, мы берем две самые большие ступени, которые в него влезают.

► Код (Скрыто) — см. интерактивную версию

```
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2 - (1 << kx) + 1][C2 - (1 << ky) + 1][kx][ky]);
int minimum = min(ans1, ans2);
```

■ **Визуализация: Разреженная таблица (1D и 2D)**

3. Декартово дерево (Treap)

Дерево поиска по ключу и Куча по приоритету.

Декартово дерево (Treap)

Tree + Heap = Treap

Строгое правило / Формула:

Treap хранит пары (x; y): для ключа x — бинарное дерево поиска, для приоритета y — двоичная куча. Пара — точка на декартовой плоскости, отсюда и название. Два базовых метода: Split и Merge.

Построение видно глазами: точки приходят как попало → сортировка по y (quicksort платит $\approx n \cdot \log n$ сравнений, counting по маленьким целым y — $O(n)$ без сравнений: разница есть, дерево получится одно) → каждая точка вставляется спуском от корня: $x \leq$ узла — влево, иначе вправо. Порядок рисования линий не важен: точки прибиты, дерево единственно. Весь процесс — в симуляторе под текстом: Собрать (сортировка на выбор, свои точки, чек-лист ) или игра  «соединить сам» — кликаешь две точки, линия проводится, а проверка мгновенно объясняет, почему нельзя: «у точки уже есть родитель», «получился бы цикл», «равные „!“ не определены». Плюс Split / Merge / Erase по шагам и разборы граблей «Миф $2k/2k+1$ » и «Поиск $\neq$ сортировка». Или включи Игру  : соедини точки сам, а судья-инварианты объяснит каждый запрещённый ход.

малый x — огромный y

X ! Й

! = y = приоритет = вершина кучи

Split(t, x_0) — таможня с двумя коридорами: зелёный «все $x \leq x_0$ », красный «все $x > x_0$ ». Спуск от корня: узел, который вместе со своим поддеревом целиком проходит по одному коридору, отправляется туда без вскрытия — досматривается только одна ветка. Поэтому split — $O(h)$, а не $O(n)$.

Merge(a, b) склеивает два дерева при условии «все x в a меньше всех x в b». Каждый шаг — одно сравнение: чей y у корня больше, тот и принимает колонну, а «стыковка» продолжается в одном его поддереве (том, что сохраняет порядок x). Итого $O(h)$ сравнений y — без перестроек и поворотов.

 Ты путаешь:

- «Отсортирую бинарным поиском»: поиск не сортирует; сравнительная сортировка — не быстрее $n \cdot \log n$.
- Дети «в массиве по $2k/2k+1$ » — только у полных деревьев; treap растёт кривым.
- «Убрать можно только самый верхний»: произвольная точка тоже вырезается — её дети сшиваются merge за $O(h)$.

Сложность — явно: поиск, вставка, удаление — $O(\log n)$ в среднем; построение — $O(n \log n)$ (сортировка + n спусков). Худший случай — приоритеты вытянулись в цепочку: $O(n)$ на операцию, $O(n^2)$ на построение.

Расшифровка X!Й: «!» — это y: маленький знак держит всё слово — убери, и останется набор букв, как treap без y останется деревом поиска-палкой. X — это x: место по алфавиту. Все стоят по алфавиту, но кричат по «!»: корень самый громкий, родитель всегда громче детей. Равные «!» — выше тот, кто раньше заявил заявку в списке; равные x — идут влево по уставу.

► Код: split (Скрыто) — см. интерактивную версию

```
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}
```

► Код: вставка, merge, erase (Скрыто) – см. интерактивную версию

```
def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key:          # равные x – влево
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc:  # порядок задаёт y
    root = insert(root, key, pri)
```

```
def merge(a, b):                # все x в a < всех x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key):           # удалить произвольную точку
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)
    return root
```

■ Визуализация: Treap: плоскость → дерево → Split/Merge/Erase



4. Splay-дерево

Самонастраивающееся дерево: поднимаем тронутый ключ в корень.

Splay-дерево

Одной фразой

Строгое правило / Формула:

Обычное BST без полей балансировки. После каждого обращения — $\text{Splay}(x)$: серия поворотов поднимает x в корень. Единственный инструмент — поворот.

Одна операция

может стоить $O(n)$

гарантии на отдельный запрос нет

Любые m операций

стоят $O(m \cdot \log n)$

амортизированное $O(\log n)$ на операцию

Память

$O(n)$: ключ + 2 ссылки

легче AVL и красно-чёрного

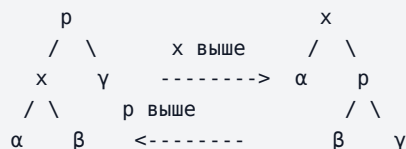
От какой боли лечит: вставь ключи по возрастанию — и BST выродится в список, поиск станет $O(n)$. AVL и красно-чёрные выбирают профилактику: хранят высоту/цвет и чинят дерево сразу. Splay выбирает лечение по факту: ничего не хранит, дереву разрешено временно быть кривым — но каждый пройденный путь заодно распрямляется.

Стопка документов: понадобился лист — вытаскиваешь и, поработав, кладёшь сверху. Сортировку не ведёшь, но через неделю самое нужное само оказывается в верхних сантиметрах, а забытое утонуло. «Взял → положил наверх».

Идёшь длинной кривой дорожкой — и после каждого прохода она сама укорачивается примерно вдвое. Первый проход дорогой, зато оплачивает следующие. Раз в год не ходишь — экономии нет; это честная слабость структуры.

Единственный кирпичик — поворот

Поворот меняет местами узел x и его родителя p . Одно движение, три перевешенные ссылки. Среднее поддерево β (между x и p) — единственный «груз», который меняет хозяина: его диапазон ключей одинаково влезает и под x , и под p .



Обход слева-направо Д0: α, x, β, p, y . ПОСЛЕ: α, x, β, p, y — тот же. Поэтому сломать дерево поиска поворотами невозможно в принципе. В симуляторе ниже каждый поворот разложен на кадры, β подсвечивается в момент «отстёжки», а врезка «Механика одного поворота» держит эту схему перед глазами.

Splay(x): крутим, пока x не станет корнем

Смотрим на тройку x (поднимаем), p (родитель), g (дед) и выбираем случай:

Случай · Как узнать · Что крутим ПЕРВЫМ · Итог

Zig · деда нет, p — корень · сам x (один поворот) · x — корень, конец

Zig-Zig · x и p с одной стороны (линия) · p вокруг g, потом x · ветка укоротилась вдвое

Zig-Zag · x и p с разных сторон (змейка) · x вокруг p, потом x вокруг g · p и g стали двумя детьми x

И по кругу: определили случай → 1-2 поворота → x ближе к корню → снова смотрим на тройку. Zig случается только последним.

△ Любимый вопрос на экзамене: Zig-Zig — это НЕ «два раза Zig»

наивно (x, потом x): 60→40→20 стало 20→60→40 — бамбук просто перевернулся, глубина та же
Zig-Zig (p, потом x): 60→40→20 стало кустом 20(40(60),...)? нет: 20 сверху, ветка укоротилась ВДВОЕ

Мнемоника: линия — крути СВЕРХУ вниз, змейка — СНИЗУ вверх. Вся разница в коде: rotate(p); rotate(x); против rotate(x); rotate(x); — одна строчка, а без неё амортизации не будет.

Полный прогон: бамбук 40 → 30 → 20 → 10, ищем 10. Спуск 4 узла, затем Zig-Zig(Zig-Zig) — 10 стала корнем, и следующий запрос к ней — ноль сравнений; на длинном бамбуке каждый такой прогон сжимает путь примерно вдвое (высота 8 → 6, 14 → 9 — проверь в симуляторе). Мы не только достали ключ, но и починили дерево по пути: все, мимо кого проходили, подтянулись к корню примерно вдвое. Отсюда амортизация: дорогой запрос нельзя пройти много раз подряд — путь кончается, и удешевляется (строго — методом потенциала: сумма log-размеров поддеревьев падает ровно на столько, сколько поворотов заплатили; отсюда же теорема о статической оптимальности — splay не хуже идеального статического дерева под твою последовательность запросов).

□ Ты путаешь:

- Zig-Zig ≠ Zig+Zig: порядок (верхнее ребро первым) сплющивает линию вдвое, «два зига» — просто переворот.
- «При повороте между A и B ничего нет» — есть: среднее поддерево β переезжает от низа к верху; в симуляторе оно подсвечивается в кадре «отстёжка».
- «Splay всегда балансирует» — одиночная операция может УВЕЛИЧИТЬ высоту; гарантия только амортизированная.

Сложность — явно: поиск/вставка/удаление — амортизированное $O(\log n)$; одиночная операция — до $O(n)$; n вставок подряд — амортизированное $O(n \log n)$.

Все операции — через один splay: find — спуск + splay найденного (нет ключа — splay последнего на пути); insert — BST-вставка + splay нового; erase — splay удаляемого, корень выкинули, у максимума левой половины не будет правого сына — туда вешаем правую. Сильные стороны: простейший код, минимум памяти, самоподстройка под «горячие» ключи, дешёвые split/merge. Слабые: нет гарантии на отдельную операцию, не годится для реального времени, дерево меняется даже при чтении (плохо для многопоточности).

► Код: rotate + splay + find (Скрыто) – см. интерактивную версию

```
# узел: { key, left, right, parent }

def rotate_up(x):
    # x встаёт на место родителя
    p = x.parent
    g = p.parent
    if p.left is x:
        # правый поворот
        p.left = x.right
        if x.right: x.right.parent = p
        x.right = p
    else:
        # левый (зеркально)
        p.right = x.left
        if x.left: x.left.parent = p
        x.left = p
    p.parent = x
    x.parent = g
    if g:
        if g.left is p: g.left = x
        else: g.right = x

def splay(x):
    # поднять x в корень
    while x.parent:
        p, g = x.parent, x.parent.parent
        if g is None:
            # ZIG
            rotate_up(x)
        elif (g.left is p) == (p.left is x):
            # ZIG-ZIG: ВЕРХНЯЯ пара первой
            rotate_up(p)
            rotate_up(x)
        else:
            # ZIG-ZAG: НИЖНЯЯ пара первой
            rotate_up(x)
            rotate_up(x)
    return x

def find(root, key):
    # поиск: спуск + splay ВСЕГДА
    cur, last = root, None
    while cur:
        last = cur
        if key == cur.key: break
        cur = cur.left if key < cur.key else cur.right
    return splay(last) if last else None
```

Вся разница между случаями – «rotate(p); rotate(x);» против «rotate(x); rotate(x);». Одна строчка – и вся амортизация.

■ **Визуализация: Splay: механика → большое дерево → собери сам**

5. Двумерная матрица префиксных сумм

Сжатие суммы подматрицы за $O(1)$

Двумерная матрица префиксных сумм

Формула включений-исключений

Строгое правило / Формула:

$$P[i][j] = P[i-1][j] + P[i][j-1] - P[i-1][j-1] + M[i][j]$$

$$\text{Сумма Rectangle}(x1...x2, y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]$$

Представь большой лист картона (площадь от (0,0) до (x2, y2)). Чтобы вырезать из него маленький целевой прямоугольник в центре, тебе нужно отрезать кусок слева и кусок сверху. Но когда ты отрезаешь их, левый верхний угол отрезается дважды! Поэтому его площадь нужно прибавить обратно.

6. Графы. Представление графов, DFS, BFS

Представление графов и базовые обходы

Графы. Представление, DFS, BFS

Матрица vs Списки смежности

Сложность хранения и обхода:

Матрица: $O(V^2)$ памяти. Обход за $O(V^2)$.

Списки смежности: $O(V + E)$. Обход за $O(V + E)$.

Матрица — таблица всех жителей Земли против всех. 99.9% таблицы пусто! Выглядит глупо, но проверка "знает ли Вася Петю" мгновенна.

Списки — записная книжка в телефоне. У каждого записано только те, кого он знает.

Оптимально для почти всех задач.

Два короля обходов: DFS и BFS

Что делает: Жадный алгоритм. Он прёт вперёд (вглубь) до упора. Берёт первую попавшуюся связанную непосещённую вершину (например, минимальную по номеру или по алфавиту) и идёт туда. Как только тупик — делает шаг назад по своим следам (рекурсивно) и пробует другой путь.

- Где используется:
- Топологическая сортировка (Билет 10)
- Поиск мостов и точек сочленения (Билет 11, 12)
- Сильно связанные компоненты Косарайю (Билет 10)
- Поиск циклов и просто проверка связности.

Что делает: Концентрические круги (слои). За один «ход» посещает ВСЕХ соседей. Потом всех соседей соседей. Работает через Очередь (FIFO). Продвигается медленно и ровно во все стороны, как круги по воде или лесной пожар.

- Где используется:
- Кратчайший путь в Невзвешенном графе (лабиринт)
- Алгоритм Ахо-Корасик (сборка автоматов)
- Алгоритм Форда-Фалкерсона / Диница (Сложная теория)
- Двухнаправленный поиск для ускорения Дейкстры.

► Код обходов C++ (Основа) — см. интерактивную версию

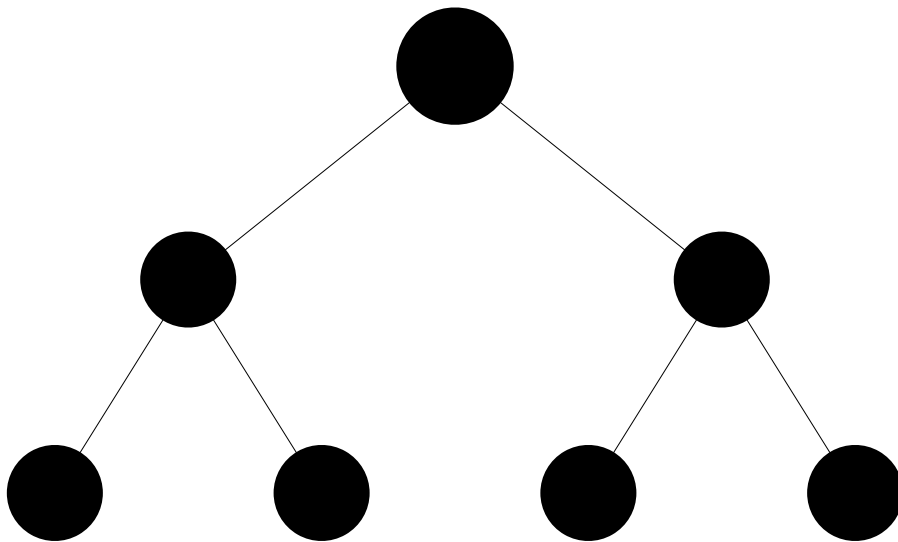
```
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}
```

```
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
```

■ Визуализация: DFS и BFS на графе



7. Графы. Планарные. Покраска

Планарность и покраска

3.1 Формула Эйлера

Формула Эйлера:

$$V - E + F = 2$$

V — вершины

E — рёбра

F — грани (включая внешнюю)

Следствие: Для планарного графа с $V \geq 3$: $E \leq 3V - 6$.

Если ребер больше — граф гарантированно непланарен. Ребра будут пересекаться.

У K_5 : $V=5$, $E=10$. Проверяем $E \leq 3V-6$: $10 > 9$ — БАХ, непланарен!

Все 5 вершин соединены со всеми. На плоскости это звезда с пересечениями.

Теорема Куратовского: K_5 — эталон непланарности.

Если внутри графа спрятан K_5 — забудь о плоской раскладке.

Три дома хотят соединиться с тремя колодцами (каждый дом — с каждым колодцем).

Нарисовать без пересечений невозможно. Это $K_{3,3}$.

У него $V=6$, $E=9$. Формула $E \leq 3V-6$ даёт $9 \leq 12$ — проходит, но он всё равно непланарен!

Почему? Потому что у двудольного графа минимальный цикл — 4, а не 3.

Отсюда более строгое ограничение: $E \leq 2V - 4$.

$9 > 8$ — попался!

► Доказательство непланарности K_5 (Скрыто) — см. интерактивную версию

Доказательство через формулу Эйлера:

Пусть K_5 планарен. $V = 5$, $E = 10$.

$F = E - V + 2 = 10 - 5 + 2 = 7$ граней.

Каждая грань ограничена минимум 3 ребрами (простых циклов длины 1-2 нет).

Сумма длин граней = $2E = 20$.

Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ — ПРОТИВОРЕЧИЕ!

Значит, K_5 не может быть планарным. Теорема доказана.

7.2 Покраска графов

Строгое правило / Формула:

$\chi(G)$ — минимальное число цветов, при котором никакие две смежные вершины не совпадают

Красим карту регионов: соседние регионы не должны быть одного цвета, иначе границы не видно.

Регион = вершина, общая граница = ребро. Вопрос «сколько цветов хватит?» — это и есть $\chi(G)$.

Для плоской карты (планарный граф!) ответ всегда: четырёх красок достаточно — теорема о четырёх красках (Аппель и Хакен, 1976, перебор на компьютере: 1936 «неудобных» конфигураций).

- Жадная раскраска: идём по вершинам $v = 0 \dots n-1$, красим в минимальный цвет, свободный у соседей. Дал $\leq \Delta + 1$ цветов (Δ — максимальная степень). Порядок вершин важен!

- $\chi(G) = 2 \iff$ граф двудолен (нет нечётных циклов). $K_{3,3}$ — двудольный, ему хватит 2 цветов, хоть он и непланарен.
- Планарный $\Rightarrow \chi \leq 4$ (теорема о четырёх красках); тривиальные оценки: $\chi(K_n) = n$, $\chi(\text{цикла чётной длины}) = 2$, $\chi(\text{цикла нечётной длины}) = 3$.
- Теорема Брукса (не путать с жадной!): для связного графа, который не полный и не нечётный цикл, $\chi(G) \leq \Delta$.

Почему раскраска «сложная»: проверить раскраску легко (пробежались по рёбрам — полином), а найти раскраску в k цветов — NP-полная задача.

Даже «раскрасима ли карта в 3 цвета?» — NP-полна. Это мостик к билету 24: задача из NP, проверка быстрая, поиск — нет.

■ **Визуализация: Планарность: $K_5/K_{3,3}$ + раскраска**

8. Графы. Компоненты связности

Нахождение кусков графа

Компоненты связности

Поиск кусков графа

Строгое правило / Формула:

Запускаем DFS/BFS от каждой непосещенной вершины. Цикл внешних запусков дает количество компонент.

Ты десантируешься на любой кусок земли. Красишь его краской. Затем смотришь на карту — остались ли серые (неисследованные) зоны? Если да — прыгаешь туда и красишь второй остров. Сколько раз пришлось прыгать через океан — столько и компонент.

■ Визуализация: Компоненты связности: BFS-разметка



9. Графы. Топологическая сортировка

Разложение задач по порядку выполнения

Топологическая сортировка

Квест-цепочка

Строгое правило / Формула:

DFS с сохранением вершин в стек В МОМЕНТ ВЫХОДА (post-order). Вывод: стек задом-наперед.

У тебя есть предметы. "Матан 2" нельзя взять, если не сдал "Матан 1".

Топологическая сортировка — это расписание, которое гарантирует, что ты не встретишь предмет, пререквизиты которого ты еще не прошел.

Не работает на графах с циклами. "Чтобы устроиться на работу нужен опыт. Чтобы получить опыт, нужна работа". Алгоритм выявит цикл и скажет, что отсортировать невозможно.

► Душный мод: Код на C++ (Скрыто) — см. интерактивную версию

Обход в глубину с разворотом списка:

```
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int to : adj[v]) {
        if (!visited[to]) {
            dfs(to);
        }
    }
    ans.push_back(v); // Добавляем в момент ВЫХОДА (post-order)
}

void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем стек
}
```

■ **Визуализация: Топологическая сортировка**

-

>	(0)	Мат	1	(1)	(2)	(3)	и	(4)	Ста	В	DFS	Сер	Чер	Шаг	1	из	0	Наз	Впе
		ан							тус		:	ые	ные					ад	ред

10. Графы. Компоненты сильной связности

Разбиение орграфа на макро-вершины. Алгоритм Косарайю.

Компоненты сильной связности (SCC)

10.1 Максимальный клуб взаимопомощи

Строгое определение:

Компонента сильной связности (SCC) — это максимальное подмножество вершин ориентированного графа, в котором существует путь из любой вершины в любую другую.

«Это просто цикл?» — Нет! SCC — это максимальная группа, где каждый может прийти до каждого.

- Если у тебя есть цикл $A \rightarrow B \rightarrow C \rightarrow A$, это одна SCC.
- Если ты добавишь вершину Г и стрелки $B \rightarrow Г$ и $Г \rightarrow A$, то теперь все четыре вершины — одна большая SCC.
- Правило: Если два цикла имеют хоть одну общую вершину — они сливаются в одну огромную SCC!

Сильная связность — это "клуб сплетников". Если Алиса расскажет секрет Бобу, Боб — Виктору, а Виктор — Алисе, то они в одной SCC. Добавить курьера Гошу, который слышит от Виктора, а сливает Алисе — и Гоша тоже в клубе. Из клуба инфа может уйти наружу (к директору), но обратно в клуб уже не вернется (иначе директор стал бы частью клуба).

SCC (Билет 10) склеивает орграфы в большие "монолитные комки", превращая весь граф в DAG (пользу).

Точки сочленения (Билет 12), наоборот, живут в неориентированных графах и показывают хрупкость монолита. Вырвешь точку сочленения — и "комната" распадется на разрозненные куски (разрыв связности).

► Душный мод: Код Алгоритм Косарайю (Скрыто) — см. интерактивную версию

Два прохода DFS:

- Запускаем DFS на исходном графе. По выходе из вершины (post-order) добавляем её в список order.
- Разворачиваем этот список (Topological Sort).
- Переворачиваем все ребра в графе (Транспонированный граф).
- Идем по order: если вершина не посещена, запускаем от неё DFS. Все, до кого дойдем — это одна SCC!

■ Визуализация: Компоненты сильной связности (Косарайю)

> 0 1 2 3 4 обх и сте СТЕ Вых (DF 1 пус ЛОГ Шаг 1 из 0 Наз Впе
ода к К ОДА S то то 1 из 0 ад ред

> Назад Вперед

11. Графы. Мосты

Мосты: Алгоритм + Код

2.1 Алгоритм Тарьяна

Ключевые массивы:

$tin[v]$ — время захода DFS в вершину v .

$low[v]$ — минимальный tin , достижимый из поддеревя v (через обратные рёбра).

$low[u] > tin[v]$

(v — родитель, u — сын в DFS-дереве)

Если из сына u и всех его потомков нельзя попасть в предка v (кроме ребра (v,u)) — это мост.

Единственная ниточка. Порвётся — граф распадётся.

Представь, что ты альпинист. Ты спускаешься в пещеру (вершина v) к напарнику (вершина u).

Из пещеры u нет других выходов — только та самая дыра, через которую ты спустился.

Если верёвка (ребро $v-u$) оборвётся — напарник навсегда замурован. Это и есть мост.

Если бы из пещеры u был чёрный ход вверх (обратное ребро) — напарник бы выбрался. И тогда это не мост.

Граф — это кровеносная система. Ребро — сосуд. Если перерезать один сосуд, но кровь может пойти по другому пути (коллатерали) — это не мост, живём.

Если же перерезать единственную артерию, ведущую к органу, и альтернатив нет — орган отмирает. Вот это мост.

Алгоритм DFS ищет такие «критические артерии».

► Полный код на C++ (Скрыто) — см. интерактивную версию

Классическая реализация — $O(V + E)$. Каждая вершина и ребро — константное число проходов.

```

#include <vector>
#include <algorithm>

using namespace std;

int n; // число вершин
vector<vector<int>> adj; // список смежности
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++; // устанавливаем время входа

    for (int to : adj[v]) {
        if (to == p) continue; // не идём назад к родителю

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v); // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}

```

Запусти визуализатор в симуляторе — там каждая строка кода подсвечивается по шагам!

■ Визуализация: Мосты: DFS + tin/low

12. Графы. Точки сочленения

Точки сочленения (Хрупкость)

12.1 Бутылочное горлышко графа

Строгое правило (через DFS Тарьяна):

Точка сочленения (Cut Vertex) — вершина неориентированного графа, удаление которой увеличивает число компонент связности.

$low[to] \geq tin[v]$

(Из сына "to" нельзя подняться СТРОГО выше "v")

Важно: Это работает только в неориентированных графах! Если мосты рвут связи между островами, то мосты крепятся к точкам сочленения. То есть, все точки вокруг мостов — это точки сочленения (кроме случаев, когда берег - это тупик со степенью 1).

Представь город, где два района соединены только одной площадью (перекрестком). Если на этой площади случится авария и её перекроют (удалят вершину) — районы будут полностью отрезаны друг от друга. Эта площадь — точка сочленения. Это про хрупкость системы.

У тебя несколько компьютеров в одном кабинете подключены к свитчу А, а в другом — к свитчу Б. И эти свитчи соединены кабелем (мостом). Сами свитчи — это точки сочленения. Сгорел свитч А — весь кабинет номер один выпал из сети, граф распался!

► Душный мод: Код и корень DFS (Скрыто) — см. интерактивную версию

Особый случай: Корень дерева DFS.

Для стартовой вершины обхода правило $low[to] \geq tin[v]$ выполняется ВСЕГДА (потому что выше неё прыгать некуда). Поэтому для неё отдельное правило: корень является точкой сочленения тогда и только тогда, когда у него больше 1 независимого ребенка в дереве DFS.

```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
```

□ Взаимосвязь с Компонентами сильной связности (SCC) из Билета 10

Это глубокий дуальный мост между ориентированными и неориентированными графами:

- Точки сочленения (Билет 12) определены на неориентированных графах и показывают физическую уязвимость связей — вынь одну вершину, и весь континент распадется на изолированные осколки.
- SCC (Билет 10) склеивают ориентированные циклы в единые независимые "конгломераты".
- Как точка сочленения рушит SCC? (Конденсация графа) Если любой орграф максимально сжать по алгоритму Косарайю, свернув каждую SCC в одну супер-вершину, мы получим ациклический граф (DAG). Если мы снимем направление с его ребер (сделаем неориентированным), то любая точка сочленения в этом полученном скелете — это и есть критическая SCC! Если удалить эту супер-вершину, то рухнет глобальный маршрут между другими клубами сильной связности. Одно слабое звено изолирует целые касты SCC навсегда.

■ Визуализация: Точки сочленения

>	0	1	2	3	4	5	6	Стату	Точка	Мост	ЛОГ
								с			

13. Графы. Эйлеров цикл (и эйлеров путь)

Эйлеров цикл и путь: различие

1.1 Определения

Строгое правило:

Эйлеров путь — проходит все РЁБРА ровно один раз. Вершины повторять можно.

Эйлеров цикл — то же самое + замкнутый (начало = конец).

Признак существования:

Путь: ровно 2 вершины с нечётной степенью (начало и конец).

Цикл: ВСЕ вершины чётные.

Билет 13 (Различие пути и цикла):

- Цикл: Вышел из дома и вернулся домой. Все степени чётные (вошёл-вышел-вошёл-вышел).

- Путь: Вышел из общаги в пивбар. Ровно 2 нечётные вершины (старт и финиш).

Ты вышел из дома (нечётная) , прошёлся по всем улицам города (по каждой ровно раз, это же Эйлер), и усталый пришёл в бар (вторая нечётная) .

Домой вернуться не смог, потому что ноги не идут. Вот и путь. Начало не равно концу.

Гамильтон — это болезнь. Синдром Гамильтона: ты одержимо хочешь посетить каждую вершину ровно раз и вернуться домой.

Ты обходишь все бары (вершины) ровно по разу. Какие улицы при этом топтать — плевать.

Главное — зайти и выйти, не заходя дважды. Это NP-полная хрень.

Никто не знает, как решать быстро. Если скажешь, что придумал алгоритм — ты либо Цукерберг, либо врешь.

Запомни: Гамильтон = Госпитализация (тебя упекут за NP-полноту).

► Душный режим: Формальное доказательство (Скрыто) — см. интерактивную версию

Теорема Эйлера:

Связный граф содержит эйлеров цикл $\Leftrightarrow$ степень каждой вершины чётная.

Эйлеров путь (не цикл) $\Leftrightarrow$ ровно 2 вершины нечётной степени (это старт и конец).

Почему Гамильтон — NP-полный:

Задача коммивояжёра (TSP) — это взвешенная версия гамильтонова пути.

TSP — NP-трудная: полиномиальный алгоритм для неё означал бы $P = NP$.

В общем, не парься. Просто запомни ассоциацию: Гамильтон = больничка.

■ Визуализация: Эйлеров путь и цикл

14. Графы. Поиск кратчайшего пути. Дейкстра

Дейкстра (Dijkstra)

Суть алгоритма

Строгое правило / Формула (Релаксация):

if $d[v] > d[u] + w(u,v)$ then $d[v] = d[u] + w(u,v)$

Дейкстра — это оптимист. Он верит, что все дороги имеют положительную стоимость (нельзя поехать и заработать). Он всегда выбирает ближайший доступный город и фиксирует кратчайший путь до него.

Если в графе есть ребра с отрицательным весом Дейкстра сломается, так как он жадный и не умеет пересматривать уже "зафиксированные" вершины.

■ Визуализация: Дейкстра



15. Графы. Поиск кратчайшего пути. Форд-Беллман

Форд-Беллман

Жизнь с отрицательными весами

Строгое правило / Формула:

Повторять релаксацию ВСЕХ рёбер ровно (V-1) раз.

Форд-Беллман — это параноик. Он не верит, что нашел кратчайший путь, и перепроверяет ВСЕ возможные дороги V-1 раз подряд. Зато он может работать с отрицательными весами!

Если запустить проверку в V-й раз, и путь вдруг станет ЕЩЕ короче — значит мы попали во временную петлю.

■ Визуализация: Форд—Беллман



16. Графы. Поиск кратчайшего пути. Флойд

Флойд-Уоршелл (Динамика на промежуточных вершинах)

Флойд не считает шаги — он считает разрешенные вершины

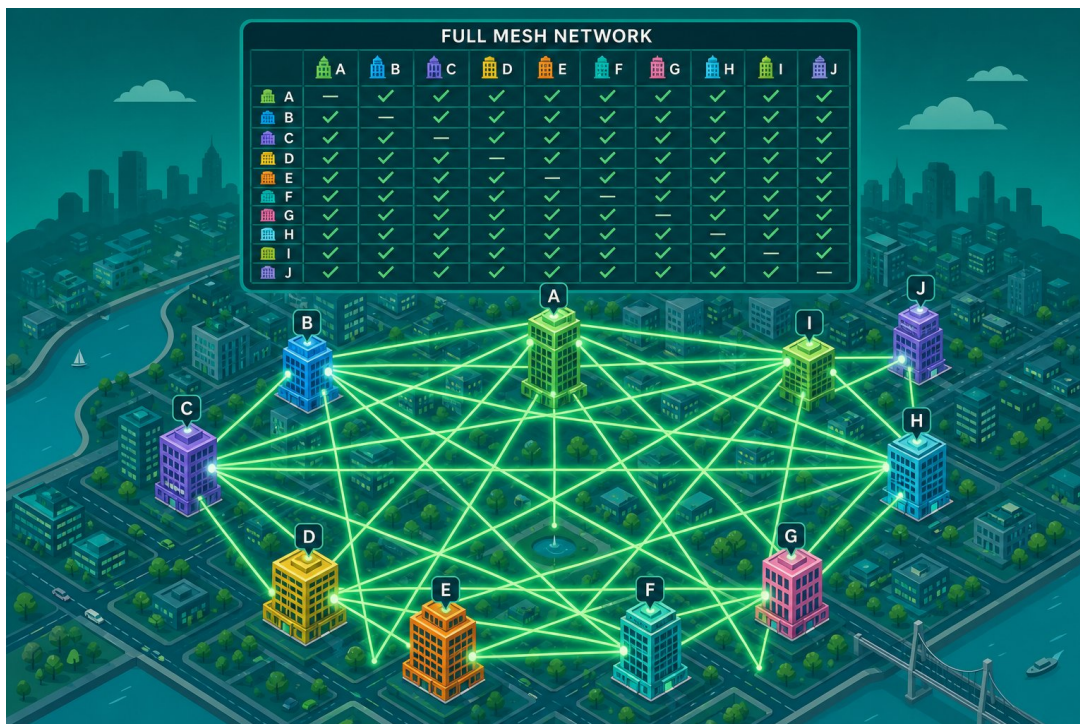
Формула (выбей на лбу):

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

Главный герой — внешний цикл for k from 1 to N . На шаге k ты спрашиваешь: «Станет ли путь от i до j короче, если я разрешу себе проходить через вершину k?»

- Шаг k=0: Ты знаешь только прямые дороги между городами (нет пересадок).
- Шаг k=1: Разрешаем пересадку через город 1. Можно ли срезать путь i -> j через путь i -> 1 -> j ?
- Шаг k=2: Разрешаем пересадки через 1 и 2. Срез i -> 2 -> j (при этом внутри этих путей уже могут быть пересадки через 1).
- ...
- Шаг k=N: Ты проверил все возможные «крюки» через все вершины. Если путь через k короче, обновляем таблицу!

■ Визуализация: Флойд—Уоршелл



17. Графы. Поиск кратчайшего пути. Ускорения (алгоритм Джонсона)

Фокус с отрицательными ребрами (Джонсон)

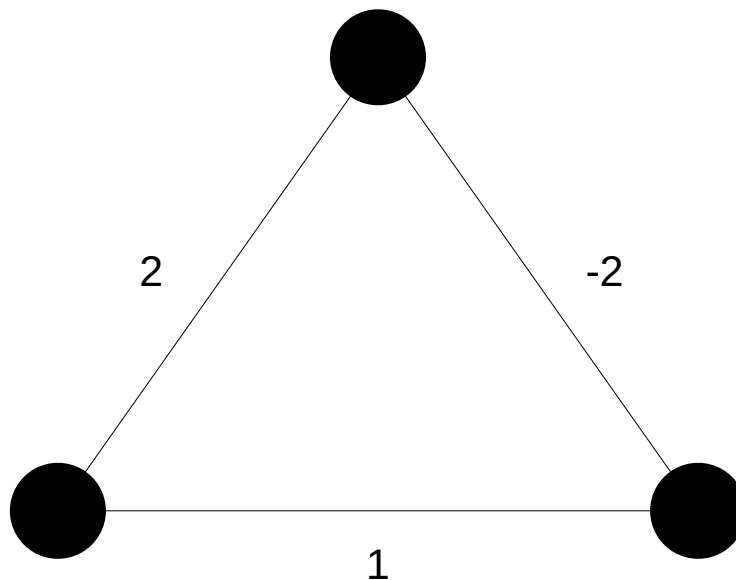
Как перестать бояться минусов и полюбить Дейкстру

$$w'(u, v) = w(u, v) + h(u) - h(v)$$

Дейкстра (Билет 14) быстрая, но плачет от отрицательных весов. Форд-Беллман (Билет 15) минусов не боится, но работает очень медленно. Мы хотим сделать Джонсона : переделать веса всех ребер так, чтобы они стали ≥ 0 , но кратчайшие пути остались теми же.

- Добавляем фиктивную вершину . Проводим от неё ребра нулевого веса до всех остальных вершин графа.
- Запускаем от неё медленного Форда-Беллмана один раз. Он найдет кратчайшие расстояния — это будут наши «потенциалы» $h(v)$.
- Меняем старые веса: новое ребро равно старый_вес плюс потенциал_начала минус потенциал_конца.
- Магия! Все веса теперь ≥ 0 . Старые кратчайшие пути остались кратчайшими («телескопическая сумма» — внутренние потенциалы сокращаются).
- Теперь смело запускаем быструю Дейкстру N раз (от каждой вершины) и не боимся!

■ Визуализация: Алгоритм Джонсона



18. Графы. Остовное дерево. Краскал

Краскал (Kruskal)

Сортировка + DSU

Строгое правило / Формула:

1. Сортируем все рёбра по весу.
2. Берём минимальное. Если оно не замыкает цикл (проверяем через DSU) - берем в ответ.

Мы хотим связать все города интернетом. Краскал говорит: "Давайте тупо строить с самых дешевых дорог в стране". Он строит кучу независимых кусочков по всей стране, пока в конце концов они все не сольются в единую сеть.

■ **Визуализация: Краскал и DSU**

19. Графы. Остовное дерево. Прима

Прима (Prim)

Растущее облако

Строгое правило / Формула:

Поддерживаем множество посещенных. На каждом шаге "откусываем" самое легкое ребро, которое торчит из посещенных в непосещенные.

Если Краскал скачет по всей карте, то Прима ведет себя как ползучая грибница (или вирус). Мы стартуем из одного города, и "щупальце" всегда захватывает ближайший, самый дешевый соседний несвязанный город. Сеть растет только из одного связного куска.

■ **Визуализация: Прим (плесень растёт от старта)**

20. Графы. Остовное дерево. Борувка

Борувка (Boruvka)

Параллельное слияние

Строгое правило / Формула:

Каждая компонента связности одновременно находит самое дешевое ребро наружу и сливается с соседом.

Сначала все города - отдельные племена. В 1-й день племя шлет гонца к самому близкому племени для объединения. К вечеру племена объединились в царства (стало в 2 раза меньше компонент). На 2-й день уже царства шлют гонцов к ближайшему чужому царству. За логарифм дней образуется Федерация интернета!

■ **Визуализация: Борувка (коллективизация)**

21. Алгоритм Кнута-Морриса-Пратта (префикс-функция)

Префикс-функция (π) — Взгляд назад

Фундамент поиска КМП

Строгое правило / Формула:

$\pi[i]$ — максимальная длина такого префикса строки, который также является её суффиксом, оканчивающимся в позиции i .

Мы идём по строке слева направо. В каждой позиции i мы спрашиваем: «Какой самый длинный кусок из начала всей строки СЕЙЧАС закончился под моим пальцем?»

Представь, ты ищешь в тексте слово `abacaba`. Ты дошел до `abacab`, и следующая буква — `x`. Тупой алгоритм начнет искать заново со второй буквы. КМП посмотрит и скажет: «Спокойно, у нас в конце было `cab`, а это начало нашего слова! Не надо перечитывать, мы уже на 3-й позиции!». Это поиск за $O(N)$ без возвратов.

Для LLM, которая стримит токены по одному, это крутой способ искать запрещенку, не перелопачивая весь контекст на каждом шаге. Если мы частично совпали с запрещённым словом, но потом пришёл не тот токен, π -функция мгновенно говорит нам, с какого места этого слова мы можем продолжить поиск, не возвращаясь назад в потоке!

► Пример вычисления (Скрыто) — см. интерактивную версию

Тот же пример: `abacabcd`

- `a` : Префиксов нет. $\pi[0] = 0$
- `ab` : Из начала ("`a`") ничего не совпало с концом ("`b`"). $\pi[1] = 0$
- `abc` : Опять мимо. $\pi[2] = 0$
- `abca` : О! Начало ("`a`") совпало с текущим концом ("`a`"). Длина — 1. $\pi[3] = 1$
- `abcab` : Начало ("`ab`") совпало с концом ("`ab`"). Длина — 2. $\pi[4] = 2$
- `abcabc` : Начало ("`abc`") совпало с концом ("`abc`"). Длина — 3. $\pi[5] = 3$
- `abacabcd` : "`d`" всё испортила. Кандидат — продолжить префикс длины 3 ("`abc`"): сравниваем `s[6]='d'` с `s[3]='c'` — не совпало. Откат по π : $j = \pi[2] = 0$. Теперь '`d`' против '`a`' — снова мимо. Общей части нет. $\pi[6] = 0$

► Код C++ (Скрыто) — см. интерактивную версию

```
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}
```

■ Визуализация: Префикс-функция (КМП)

22. Z-функция

Z-функция — Сдвиг и наложение

Z-функция

Строгое правило / Формула:

$Z[i]$ — длина наибольшего общего префикса между самой строкой (начинающейся с индекса 0) и её суффиксом, начинающимся с i .

Тут мы берём всю строку и «прикладываем» её к самой себе, начиная с каждой позиции i . Мы спрашиваем: «Если я начну читать строку отсюда, насколько длинный кусок совпадёт с самым началом строки?»

Это самый быстрый способ найти все вхождения подстроки в строку. Ты просто пишешь `Pattern + "$" + Text`, считаешь Z-функцию, и там, где $Z[i]$ равно длине `Pattern` — там и есть вхождение.

В LLM Z-функция может применяться для дедупликации или детекта зацикливаний. Если во время генерации вдруг $Z[i]$ (где i указывает назад на тот же текст) выстреливает в большое значение — значит, модель начала копипастить сама себя.

► Пример вычисления (Скрыто) — см. интерактивную версию

Тот же пример: `abccabacd`

- $Z[0]$ — не считается (или длина строки).
- $Z[1]$ (`bcabacd` vs `abccabacd`): Первая буква 'b' != 'a'. $Z[1] = 0$
- $Z[2]$ (`cabacd` vs `abccabacd`): 'c' != 'a'. $Z[2] = 0$
- $Z[3]$ (`abacd` vs `abccabacd`): 'a'=='a', 'b'=='b', 'c'=='c', 'd'!='d' (подожди, 'd' == 'a'? Нет, в начале 'a', тут 'd'). Совпало 3 буквы. $Z[3] = 3$
- $Z[4]$, $Z[5]$, $Z[6]$ — всё по нулям, 'b', 'c', 'd' не 'a'.

► Код C++ (Скрыто) — см. интерактивную версию

```
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
```

■ Визуализация: Z-функция

23. Алгоритм Ахо-Корасик

Алгоритм Ахо-Корасик

Множественный поиск

Основная формула:

Бор (Trie) + Суффиксные ссылки + Терминальные ссылки = Конечный автомат

Позволяет найти сразу множество слов из словаря в большом тексте за время $O(N)$ совершив всего один проход.

Представь, что мы сплели паутину (Бор) из нужных нам слов. Если нужного продолжения буквы нет — мы падаем по суффиксной ссылке ("нити страховки") в самый длинный из известных суффиксов.

Представь, что мы ищем слова "he" и "she". Если мы прочитали "she", мы одновременно нашли и "he", потому что оно спрятано внутри как матрёшка! Для этого нужны терминальные ссылки.

► Код (Скрыто) — см. интерактивную версию

```
int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) t[v].link = 0;
        else t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char c) {
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
    }
    return t[v].go[c];
}
```

■ **Визуализация: Ахо—Корасик**

> [] [] Сен [] Пря из Я Сен Вве сло в или наж и я как сет
я мое я! ди ва ми

> [] CAR x CAT x CAR x ART x [] и Все 0 ID Сим Дет Сло (di
Т го вол и ва ct)

24. Классы сложности, сведение задач

Классы сложности (P, NP, NP-hard)

Границы невозможного

Строгое правило / Формула:

P: Можно решить быстро (полином).

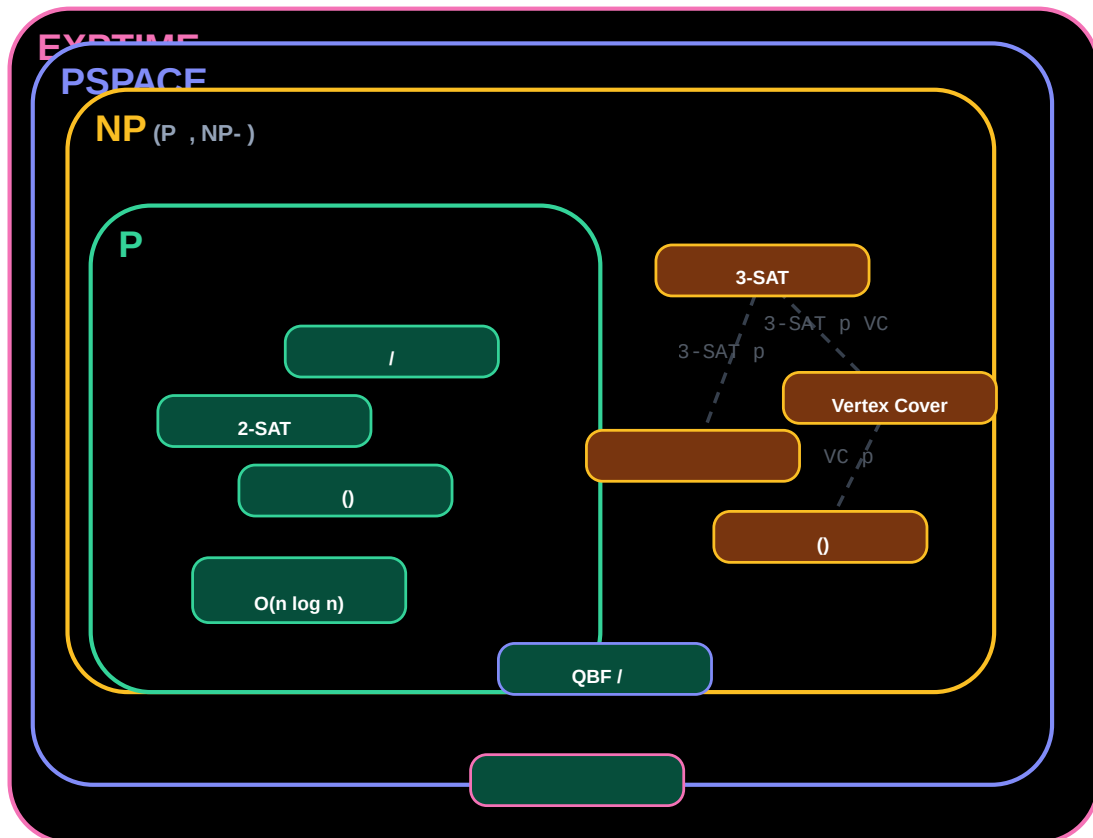
NP: Можно БЫСТРО ПРОВЕРИТЬ готовый ответ.

Сведение (Reduction) $A \leq_p B$: вход задачи A преобразуется за полином во вход задачи B; умеешь решать B — решаешь и A.

Класс P — это когда алгоритм сам берет и быстро решает пустую доску (например сортировка чисел). Класс NP — это когда алгоритм не знает как собирать большое sudoku, но если ты дашь ему заполненную сетку (ответ), он БЫСТРО (за класс P) проверит, что там нет ошибок по правилам.

Сведение A к B — это когда ты не умеешь говорить по-китайски (A), но у тебя есть отличный переводчик на английский (Сведение) и друг, болтающий по-английски (решение B). Если любая NP-задача сводится к задаче B, а сама B лежит в NP, то B — NP-полная (сосредоточение всего зла). Без « $B \in NP$ » это лишь NP-трудная!

■ Визуализация: Классы сложности: матрёшка P $\subseteq$ NP $\subseteq$ PSPACE



>	P	быстр	решает	NP	быстр	PSPACE	мало	памят
		о	м		о	Е		и